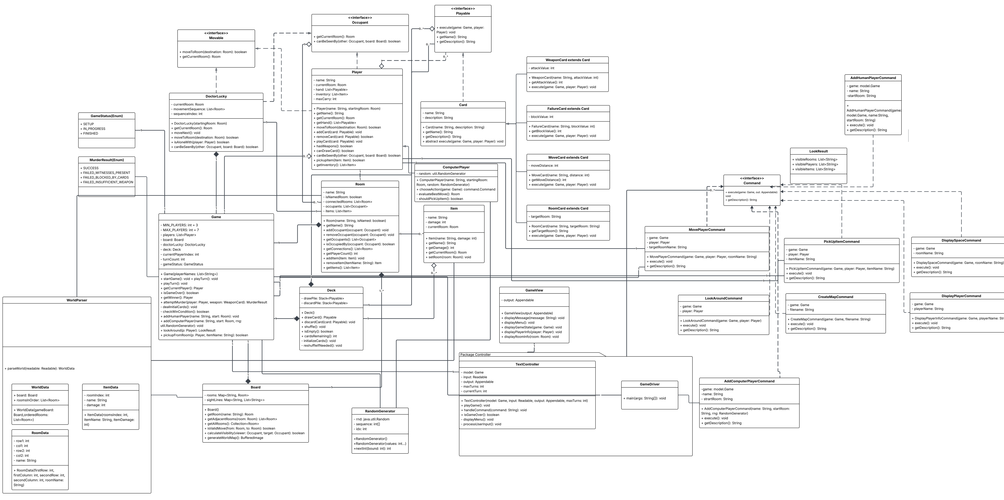

Test Plan for KillDoctorLucky

1. Game Class Test
   a. Valid game initialization
      i. Condition: Game constructor with valid player names
      ii. Example Data: ["Peter", "Will", "Alice", "Henry"]
      iii. Expected: the Game created, gameStatus = SETUP
   b. Minimum Players(Valid)
      i. Condition: Game constructor with minimum players to be 3
      ii. Example Data: ["Peter", "Will", "Alice"]
      iii. Expected: Game created successfully.
   c. Maximum Players (Valid)
      i. Condition: Game constructor with maximum players to be 7
      ii. Example Data: ["Peter", "Will", "Alice", "Henry", "Alan", "Olivia", "George"]
      iii. Expected: Game created successfully.
   d. Below Minimum Players
      i. Condition: Game constructor with 2 players
      ii. Example: ["Peter", "Henry"]
      iii. Expected: Exception thrown
   e. Above Maximum Players
      i. Condition:Game constructor with 8 players
      ii. Example:["Peter", "Will", "Alice", "Henry", "Alan", "Olivia", "George", "Jason"]
      iii. Expected: Exception thrown
   f. Start Game
      i. Condition: startGame called
      ii. Example: Game with 3 players
      iii. Expected: gameStatus changes from SETUP to IN_PROGRESS
   g. Successful Murder
      i. Condition: attemptMurder() with player alone, strong weapon
      ii. Example: Alice alone with Doctor Lucky, WeaponCard(value = 10)
      iii. Expected: MurderResult.SUCCESS, game over.
   h. Failed Murder - Witness
      i. Condition: attemptMurder() with witness present
      ii. Example: Alice, Henry and Doctor Lucky all in same room
      iii. Expected: MurderResult.FAILED_WITNESS_PRESENT
2. Player Class Test
   a. Player Creation
      i. Condition: Player constructor with valid data
      ii. Example: name = "Alice", startingRoom = Kitchen
      iii. Expected: Player created, currentRoom = Kitchen, empty hand
   b. Valid Movement
      i. Condition: moveToRoom() to connected room
      ii. Example: Player in Kitchen, move to Dining Room
      iii. Expected: Returns true, currentRoom = Dining Room
   c. Invalid Movement
      i. Condition: moveToRoom() to unconnected room
      ii. Example: Player in Kitchen, move to Bedroom (not connected)
      iii. Expected: Return False, currentRoom unchanged
   d. Add Card to Hand
      i. Condition: addCard() with space available
      ii. Example: Player with 3 cards, add WeaponCard("Knife", 5)
      iii. Expected: hand. size() = 4, card added
   e. Hands initialization
      i. Condition: Player starts with 6 cards
      ii. Example: New Game initialization
      iii. Expected: each player's hand.size() = 6
3. Board and Room Test
   a. Get Valid Room
      i. Condition: getRoom() with existing room name
      ii. Example: getRoom("Kitchen")
      iii. Expected: Returns Kitchen room object
   b. Invalid Room
      i. Condition: getRoom() with non-existent room
      ii. Example: getRoom("InvalidRoom")
      iii. Expected: Returns null or throws exception
   c. Add Occupant to Room
      i. Condition: addOccupant() with player
      ii. Example: add Henry to Kitchen
      iii. Expected: Henry in Kitchen's occupant list
   d. Room Visibility
      i. Condition: calculateVisibility() between rooms
      ii. Example: Alice in Kitchen and Bob in Dining Room (connected)
      iii. Expected: Returns True (can see each other)
4. Doctor Lucky Test
   a. Sequential Movement
      i. Condition: moveNext() called multiple times
      ii. Example: Movement sequence [Kitchen, Dining, Living], call 3 times
      iii. Expected: Kitchen to Dining to Living to Kitchen (circle)
   b. Alone Check-True
      i. Condition: isAloneWith() with only player present
      ii. Example: Kitchen has only Alice and Doctor Lucky
      iii. Expected: doctorLucky.isAloneWith() return True
   c. Alone Check-False
      i. Condition: isAloneWith() with multiple player
      ii. Example: Kitchen has Alice, Henry and Doctor Lucky
      iii. Expected: doctorLucky.isAloneWith() return False
5. Card Test
   a. Weapon Card
      i. Condition: WeaponCard creation and execution
      ii. Example: WeaponCard("Knife", 5)
      iii. Expected: Card created attackValue = 5
   b.Failure Card
      i. Condition: FailureCard blocks murder attempt
      ii. Example: FailureCard("Bad Luck", 3) used during murder
      iii. Expected: reduce murder success by 3 points
   c.Move Card
      i. Condition: MoveCard allows extra movement
      ii. Example: MoveCard("Quick Step", 2)
      iii. Expected: Player can move up to 2 additional rooms
   d. Room Card
      i. Condition: RoomCard instant teleportation
      ii. Example: RoomCard("Kitchen Card", "Kitchen")
      iii. Expected: player instantly move to Kitchen
6. Deck Test
   a.Empty Deck Reshuffle
      i. Condition: drawCard() when draw pile empty
      ii. Example: Empty draw pile, 20 cards in discard
      iii. Expected: discard reshuffled, card drawn
   b. Draw Card
      i. Condition: drawCard() with cards available
      ii. Example: Deck with 50 cards
      iii. Expected: return card, deck size decrease
   c.Discard Card
      i. Condition: discardCard() adds to discard pile
      ii. Example: discard WeaponCard("Knife", 5)
      iii. Expected: Card added to discard pile
7. Interface Test
   a. Playable Interface
      i. Condition: All cards implement Playable
      ii. Example: WeaponCard, MoveCard treated as Playable
      iii. Expected: execute(), getName() work for all cards
   b. Movable Interface
      i. Condition: Player and Doctor Lucky implement Movable
      ii. Example: Both objects as Movable type
      iii. Expected: moveToRoom(), getCurrentRoom() work for both
8. Integration Test
   a.Game end
      i. Condition: Game ends when murder successful
      ii. Example: Alice successfully murders Doctor Lucky
      iii. Expected: gameStatus = FINISHED, Alice declared winner
   b. Multiple player interaction
      i. Condition: players interact during murder attempt
      ii. Example: Alice attempts murder, Bob plays FailureCard
      iii. Expected: Bob's card affects Alice's murder attempt
   c.Complete Turn
      i. Condition: full player turn with movement and card play
      ii. Example: Alice moves, plays MoveCard, attempts murder
      iii. Expected: all actions execute correctly, turn passes to next player
9. Controller Test
   a.Display Menu
      i. Condition: playGame() starts and displays menu
      ii. Example: TestController with valid Game
      iii. Expected: Menu displays all available commands
   b. Process Valid Command
      i. Condition: handleCommand() with valid input
      ii. Example: input "move Kitchen"
      iii. Expected: Player moves, success message displayed
   c.Process Invalid Command
      i. Condition: handleCommand() with invalid input
      ii. Example: input 'xyz123'
      iii. Expected: Error message game continues
   d.MaxTurns Limit
      i. Condition: Game reaches maxTurns
      ii. Example: maxTurns = 10, play 10 turns
      iii. Expected: Game ends, final state displayed
   e.Game Over Detection
      i. Condition: isGameOver() returns true when done
      ii. Example: After max turns or successful murder
      iii. Expected: returns true, game stops
10. Command Pattern Test
   a.MovePlayerCommand Execution
      i. Condition: execute() moves player
      ii. Example: MovePlayerCommand(game, alice, "Kitchen")
      iii. Expected: alice.getCurrentRoom() = Kitchen
   b. PickUpItemCommand Execution
      i. Condition: execute() picks up item
      ii. Example: PickUpItemCommand(game, alice, "Knife")
      iii. Expected: alice.getHand() contains Knife
   c.LookAroundCommand Execution
      i. Condition: execute() displays surroundings
      ii. Example: LookAroundCommand(game, alice)
      iii. Expected: Returns list of visible rooms/players
   d.Command Polymorphism
      i. Condition: All Commands work as Command interface
      ii. Example: Command cmd = new MovePlayerCommand(...)
      iii. Expected: cmd.execute() works correctly
11. ComputerPlayer Test
   a. Automatic Action Selection
      i. Condition: chooseAction() returns valid Command
      ii. Example: ComputerPlayer with RandomGenerator(1,2,3)
      iii. Expected: Returns MovePlayerCommand predictably
   b. Random Behavior
      i. Condition: Different RandomGenerator gives different actions
      ii. Example: Two ComputerPlayers with different seeds
      iii. Expected: Make different choices
   c. Valid Moves Only
      i. Condition: Computer never chooses invalid move
      ii. Example: ComputerPlayer in corner room
      iii. Expected: Only chooses valid neighboring rooms
12. View Test
   a. Display Message
      i. Condition: displayMessage() outputs to Appendable
      ii. Example: GameView with StringBuffer, displayMessage("Hello")
      iii. Expected: StringBuffer contains "Hello"
   b. Display Game State
      i. Condition: displayGameState() shows current info
      ii. Example: Game with 3 players, turn 5
      iii. Expected: Output shows player positions and turn count
13. RandomGenerator Test
   a. Predictable Mode
      i. Condition: Constructor with varargs returns predefined values
      ii. Example: RandomGenerator(1,2,3), call nextInt(10) three times
      iii. Expected: Returns 1, 2, 3 in order
   b. Random Mode
      i. Condition: Default constructor gives random values
      ii. Example: RandomGenerator(), call nextInt(10) multiple times
      iii. Expected: Returns different values each time
   c. Cycle Through Values
      i. Condition: Predefined values cycle when exhausted
      ii. Example: RandomGenerator(1,2), call 5 times
      iii. Expected: Returns 1,2,1,2,1